

- Fechas -

17/08 Feriado

14/09 **1° PARCIAL** → única fecha movable (capaz, en lo posible no)

21/09 Feriado

12/10 Feriado

02/11 **2° PARCIAL**

09/11 **RECUP. DENTRO DE CURSADA**

16/11 RECUP. FUERA DE CURSADA

23/11 Feriado

Para los parciales da machete para evitar errores de tipeo



```
SELECT 'hola mundo', upper('hola mundo'), lower('hola mundo'), initcap('hola mundo');
```

Express Edition

```
SELECT *  
FROM employee  
WHERE last_name LIKE 'S%'; → que su apellido empiece con S
```

```
SELECT *  
FROM employee  
WHERE (salary > 1000  
AND department_id = 30)  
OR manager_id = 7902; → separa en términos. ((salario y departamento) ó manager_id)
```

```
SELECT *  
FROM employee  
WHERE salary BETWEEN 1200  
AND 1350;
```

Recomendación: Separar con enter cada condición para detectar mejor los errores por nro de línea

```
SELECT first_name||' '||last_name, salary → Concatenar columnas, agregando espacios (ej. nombre y ap)  
FROM employee  
WHERE department_id = 10  
ORDER BY first_name; → Ordena alfabéticamente
```

```
SELECT DISTINCT name → El "distinct" no los repite  
FROM department  
WHERE NAME = 'sales'; → nombre de los depts que tiene mi compañía, sin repetir
```

Comentar línea --, Comentar texto //**

```
SELECT first_name, last_name, salary, commission, salary+commission → Generar columna nueva  
FROM employee  
WHERE commission IS NOT NULL; → nro + nro = nro, pero nro + null = null ¡cuidado!  
evitar nulls cuando hacemos cálculos a toda una columna
```

Función **NVL** → "null value"
tiene 2 variables (campo, valor)

NVL (campo, valor)

Si tiene un valor en "campo", devuelve "campo"; si "campo" es nulo, devuelve "valor".

Ejemplo: SELECT first_name, last_name, salary, commission, salary+NVL(commission, 0)
FROM employee

→ Entonces si un commission es null, no te imprime null, sino que te devuelve salary+0.

SELECT first_name, last_name, department_id

FROM employee

WHERE department_id = &department; → **Variable de sustitución**: Lo que va después del & ó : , va a ser lo que le especifiquemos en la ventana emergente.

SELECT first_name, last_name, name → "name" atributo de la tabla de departamentos

FROM employee E,

department D,

→ **Forma del profe de usar cualquier JOIN**

WHERE E.department_id = D.department_id;

SELECT SYSDATE Devuelve la fecha del motor

FROM DUAL Podemos jugar sumándole y restándole días (ej. SYSDATE +7)

SELECT *

FROM sales_order

WHERE order_date >= to_date('01/01/2010', 'DD/MM/YY') → **transforma cualquier entrada en formato fecha**, según cómo se lo especifiquemos.

SELECT to_char(SYSDATE, 'DD/MM/YYYY HH24:MI:SS') fecha → **transforma cualquier entrada en char**, según cómo se lo especifiquemos.

SELECT to_char(**TRUNC**(SYSDATE), 'DD/MM/YYYY HH24:MI:SS') fecha_trunc → **trunca** ..?
FROM DUAL;

- SELECT **TRUNC**(10124.86) → trunc sin segunda variable solo corta los decimales.
- SELECT **TRUNC**(10124.86, -1) → Sino -1 te trunca y deja el último nro. en cero, -2 los últimos dos, etc.

SELECT department_id, **COUNT**(employee_id), → **cuenta cantidad de empleados**

SUM(salary) → **suma los salarios de los empleados**

ROUND(**AVG**(salary), 2) → **promedio de los salarios y lo redondea!**

FROM employee

GROUP BY department_id;

HAVING count(1) > 3 → Condiciona lo que se muestra.

Ej: solo los departamentos que tengan más de 3 empleados

SELECT first_name, last_name, department_id, salary

FROM employee

WHERE salary > (**SELECT** avg(salary) → **SUBCONSULTA**

FROM employee)

Empleados que ganan más que el promedio de los salarios.

U1-E02 - Query

Mostrar los empleados que ganan más que su jefe.

El reporte debe mostrar el nombre completo del empleado, su salario, el nombre del departamento al que pertenece y la función que ejerce.

```
SELECT E.first_name||' '||E.last_name AS fullname,
       E.salary,
       D.name,
       J.function
FROM employee E,
     department D,
     job J,
WHERE (E.department_id = D.department_id)
      AND (E.job_id = J.job_id)
      AND (E.salary > (SELECT salary
                      FROM employee
                      WHERE employee_id = manager_id);
```

PL/SQL

UPDATE tabla
SET campo=valor
WHERE condición → mínimo: ninguna condición.

Te devuelve la cantidad de tablas que se
modificaron por coincidir con la condición:
0, 1, n, error!

DELETE
FROM tabla
WHERE condición

Te devuelve la cantidad de tablas que se eliminaron
por coincidir con la condición: 0, 1, n, error! → no puede dejar objetos huérfanos

INSERT
INTO tabla
(campo1, campo2, ..., campo n) → No es obligatorio, pero para nosotros sí. ¡Buena práctica! 👍
VALUES (valor 1, valor 2, ..., valor n)

Inserta como mínimo 1 campo y puede dar error! (*"Dar de alta"*)
(ej. por tipo de dato o foreign key)

PL/SQL	Devuelve
UPDATE	0, 1, n, error!
DELETE	0, 1, n, error!
INSERT	1, error!
SELECT campo1, campo2, ..., campon INTO variable1, variable2, ..., variablen FROM tabla WHERE condición	SOLO 1 Si da más, es error!

Hay distintos tipos de errores